

WEB SCRAPER CHROME EXTENSION

Development Milestones Plan

15 Milestones — Feature by Feature

Overview

This document breaks the entire project into 15 individual milestones, one per feature. Each milestone is self-contained — it has a clear goal, list of deliverables, files to create, and a test to confirm it is done. Complete them in order as each one builds on the previous.

How to use this plan:

- Complete milestones in order from M1 to M15
- Do not move to the next milestone until the current one passes its test
- Each milestone = one working, testable feature
- Beginner notes are included to help with tricky parts

Milestone Summary

#	Milestone	Goal	Status
M1	Chrome Extension Skeleton	Get a working Chrome extension that loads in the browser with a basic popup w...	☐ Pending
M2	Content Script Injection	Inject a script into any website the user visits so the extension can read an...	☐ Pending
M3	Element Hover Highlighting	When user activates selection mode, hovering over any element on the website ...	☐ Pending
M4	Live XPath Display on Hover	As the user hovers over any element, a tooltip shows that element's XPath in ...	☐ Pending
M5	Click to Select Fields	User clicks a highlighted element to add it as a field to scrape. A dialog ap...	☐ Pending
M6	Single Page Data Extraction	User clicks 'Start Scraping' and the extension extracts all matching data fro...	☐ Pending
M7	Live Data Preview Table	After scraping, a live table inside the popup	☐

		shows all extracted rows and co...	Pending
M8	Column Rename & Delete	User can rename any column header and delete unwanted columns from the previe...	 Pending
M9	Unlimited Scrolling Support	For infinite scroll websites (Instagram, Facebook, LinkedIn), the extension a...	 Pending
M10	Pagination — Next Button Support	For paginated websites, the extension automatically detects and clicks the Ne...	 Pending
M11	AI Auto-Detection	When extension opens on any page, it automatically identifies the most likely...	 Pending
M12	Export — CSV, Excel, JSON	User can download all scraped data in their chosen format with one click.	 Pending
M13	Template Save & Reuse	User can save their field configuration as a named template. Next time they v...	 Pending
M14	Progress Controls — Pause, Resume, Stop	During scraping, user has full control with Pause, Resume, and Stop buttons a...	 Pending
M15	Final UI Polish & Testing	Full UI cleanup, error handling, edge case fixes, and testing on 10+ differen...	 Pending

Detailed Milestone Breakdown

M1: Chrome Extension Skeleton

 **Goal:** Get a working Chrome extension that loads in the browser with a basic popup window.

 **Why it matters:** Every feature depends on this foundation. Without it nothing else can be built.

Deliverables

- manifest.json file configured with Manifest V3
- Basic popup window opens when extension icon is clicked
- Folder structure created (popup, content, background, utils, icons)
- Extension loads successfully in Chrome without errors
- Placeholder icon and extension name visible in Chrome toolbar

 **Files:** manifest.json | popup/popup.html | popup/popup.css | popup/popup.js | icons/

 **Done When:** Open Chrome → go to chrome://extensions → load extension → click icon → popup appears

 **Beginner Note:** This is the hardest mental step — everything feels new. Follow Chrome's official getting started guide. No scraping logic yet, just structure.

M2: Content Script Injection

 **Goal:** Inject a script into any website the user visits so the extension can read and interact with that page.

 **Why it matters:** The content script is the extension's 'eyes' on the website. Without it, nothing can be selected or scraped.

Deliverables

- content.js file created and injected into active tab
- Console log confirms script is running on the website
- Popup button triggers content script injection
- Script works on any website (Amazon, Shopify, etc.)
- No errors or permission issues

 **Files:** content/content.js | background/service-worker.js

 **Done When:** Visit Amazon → click extension → open browser console → see 'Content script loaded' message

 **Beginner Note:** Content scripts are separate from your popup. They run inside the website. Communication between popup and content script uses Chrome message passing — this is the trickiest beginner concept.

M3: Element Hover Highlighting

 **Goal:** When user activates selection mode, hovering over any element on the website highlights it with a visible border.

 **Why it matters:** This is the core UX of the scraper. User needs to visually see what they are selecting before clicking.

Deliverables

- Popup button 'Start Selection' activates hover mode
- Blue border appears around any element user hovers over
- Border follows the cursor as user moves across the page
- Hovering stops when user clicks or presses Escape
- Highlight does not break website layout

 **Files:** content/content.js | content/highlighter.css

 **Done When:** Click 'Start Selection' → go to any website → move mouse → elements get blue border

 **Beginner Note:** Use CSS outline (not border) for highlighting — it does not affect page layout. Inject a CSS class dynamically and remove it on mouseout.

M4: Live XPath Display on Hover

 **Goal:** As the user hovers over any element, a tooltip shows that element's XPath in real time.

 **Why it matters:** Client specifically requested this feature. Advanced users can verify exactly what they are selecting.

Deliverables

- Tooltip appears near the cursor showing full XPath
- XPath updates in real time as cursor moves to different elements
- Tooltip is clearly readable (white background, dark text)
- XPath is copyable — user can copy it from the tooltip
- Tooltip disappears when selection mode is turned off

 **Files:** content/content.js | content/highlighter.css

✓ **Done When:** Hover over a product title on Amazon → tooltip shows `//div[@class='...']` in real time

📝 **Beginner Note:** XPath generation from a DOM element: walk up the tree collecting tag names and positions. There are open-source XPath generator functions you can reuse.

M5: Click to Select Fields

🎯 **Goal:** User clicks a highlighted element to add it as a field to scrape. A dialog appears to name the field.

💡 **Why it matters:** This is how the user tells the extension what data to collect. Each click = one column in the final data.

Deliverables

- Clicking a highlighted element opens a field dialog
- User can type a name for the field (e.g. 'Product Name')
- User can select field type (Text, Link, Image, Price)
- Field is saved and shown in the popup field list
- User can add multiple fields one by one
- User can remove a field from the list

📁 **Files:** `content/content.js` | `popup/popup.html` | `popup/popup.js` | `utils/storage.js`

✓ **Done When:** Hover over Amazon product title → click it → dialog appears → type 'Product Name' → see it in field list

📝 **Beginner Note:** Store fields in Chrome storage (`chrome.storage.local`). Send messages between content script and popup using `chrome.runtime.sendMessage`.

M6: Single Page Data Extraction

🎯 **Goal:** User clicks 'Start Scraping' and the extension extracts all matching data from the current page.

💡 **Why it matters:** This is the core engine — the first time actual data gets collected. Everything before this was setup.

Deliverables

- Extension finds all repeating row containers on the page
- Extracts selected fields from each row
- Handles text, links, image URLs, and price extraction
- Shows count of items found (e.g. '47 items found')
- Handles missing elements gracefully (no crash if field not found)

- Data stored in memory ready for export

📁 **Files:** content/content.js | background/scrapper-engine.js | utils/dom-parser.js

✅ **Done When:** Select 'Product Name' and 'Price' on Amazon → click Scrape → see 48 items extracted in popup

📝 **Beginner Note:** Use `querySelectorAll()` with the saved CSS selector to find all matching elements. Loop through each one and extract the text/src/href depending on type.

M7: Live Data Preview Table

🎯 **Goal:** After scraping, a live table inside the popup shows all extracted rows and columns before export.

💡 **Why it matters:** Client needs to verify data is correct before downloading. This prevents wasted exports.

Deliverables

- Table renders inside popup showing all scraped rows
- Column headers match the field names user defined
- Table scrolls if data is large
- Updates live as more data is scraped
- Empty cells shown clearly (not hidden)

📁 **Files:** popup/popup.html | popup/popup.css | popup/popup.js

✅ **Done When:** After scraping Amazon — popup shows a clean table with Product Name and Price columns filled with real data

📝 **Beginner Note:** Build the table dynamically using JavaScript DOM manipulation. Create thead with field names and tbody with one tr per scraped row.

M8: Column Rename & Delete

🎯 **Goal:** User can rename any column header and delete unwanted columns from the preview table.

💡 **Why it matters:** Raw scraped column names are often ugly (`span.price`). User needs clean names for their export file.

Deliverables

- Double-click a column header to rename it inline
- Delete button (x) on each column removes it from table

- Changes reflect immediately in the preview
- Renamed columns appear in exported file
- Deleted columns are excluded from export

 **Files:** popup/popup.html | popup/popup.css | popup/popup.js

 **Done When:** Rename column 'span.a-price' to 'Price' → delete 'Image URL' column → export → Excel file has clean column names

 **Beginner Note:** Make column headers contenteditable='true' for inline renaming. Keep a separate fieldMap object that tracks original selector → display name.

M9: Unlimited Scrolling Support

 **Goal:** For infinite scroll websites (Instagram, Facebook, LinkedIn), the extension automatically scrolls down continuously to load all content before scraping.

 **Why it matters:** Many modern websites do not have a Next button — they load more data as you scroll. Without this, only the first visible items are scraped.

Deliverables

- Extension detects if page uses infinite scroll
- Auto-scrolls to bottom of page in steps
- Waits for new content to load after each scroll
- Stops when no new content appears (end of page)
- User can stop scrolling at any time with Stop button
- Progress shown: 'Scrolled X times, Y items found'

 **Files:** content/content.js | background/scrapper-engine.js

 **Done When:** Open Instagram explore page → click Scrape → extension scrolls automatically → more posts load → all collected

 **Beginner Note:** Use window.scrollTo() in a loop with a delay (setTimeout/setInterval). Detect end of scroll by comparing scrollHeight before and after scroll.

M10: Pagination — Next Button Support

 **Goal:** For paginated websites, the extension automatically detects and clicks the Next button, scraping each page until the last one.

 **Why it matters:** Most e-commerce sites like Amazon, eBay, and WooCommerce use pagination. Without this, user gets only page 1.

Deliverables

- Auto-detects Next button using 10+ common patterns
- Clicks Next button and waits for page to load
- Scrapes each new page and adds to existing data
- Stops when no Next button is found (last page)
- Shows progress: 'Page 3 of 12 — 287 items collected'
- Pause and Stop buttons work during pagination
- User can manually specify Next button selector if auto-detect fails

 **Files:** background/scrapper-engine.js | content/content.js | popup/popup.js

 **Done When:** Open Amazon search results → click Scrape → extension scrapes page 1 → clicks Next → scrapes page 2 → continues automatically

 **Beginner Note:** Use MutationObserver to detect when a new page has loaded after clicking Next. Check for URL change or DOM change to confirm page transition.

M11: AI Auto-Detection

 **Goal:** When extension opens on any page, it automatically identifies the most likely data table or repeating list and pre-selects the fields — no manual clicking needed.

 **Why it matters:** This matches Instant Data Scraper's key feature. Makes the tool beginner-friendly for users who do not know what to click.

Deliverables

- Extension scans page for repeating HTML patterns automatically
- Highlights the detected data region visually
- Pre-populates field list with auto-detected columns
- User can accept auto-detection or switch to manual selection
- Works on tables (HTML table tags) and div-based lists
- Confidence score shown (e.g. 'Table detected — 94% confidence')

 **Files:** content/content.js | utils/dom-parser.js

 **Done When:** Open any data table website → click extension → fields are already pre-filled without any clicking

 **Beginner Note:** Start with detecting HTML table tags (easiest). Then detect repeating div/li/article patterns by finding elements with the same class repeated 3+ times.

M12: Export — CSV, Excel, JSON

 **Goal:** User can download all scraped data in their chosen format with one click.

 **Why it matters:** Data is useless without export. This is the final step that delivers value to the user.

Deliverables

- Export to CSV with proper comma escaping
- Export to Excel (.xlsx) with formatted headers
- Export to JSON with full data structure
- Column selection before export (choose which columns to include)
- Custom filename input
- One-click download — file appears in browser Downloads

 **Files:** `utils/exporter.js` | `popup/popup.js` | `lib/xlsx.min.js`

 **Done When:** Scrape 100 Amazon products → click Export → choose Excel → file downloads → open in Excel → data is clean and formatted

 **Beginner Note:** For CSV use `Blob + URL.createObjectURL()`. For Excel use the `xlsx.js` library (SheetJS) — it is free and well documented. For JSON use `JSON.stringify()`.

M13: Template Save & Reuse

 **Goal:** User can save their field configuration as a named template. Next time they visit the same site, the template is suggested automatically.

 **Why it matters:** Users who scrape the same sites repeatedly should not redo setup every time. This saves hours over time.

Deliverables

- Save current field config as a named template
- Templates stored in Chrome local storage
- Template list shown in extension options
- Auto-suggest template when user visits a known URL
- Load template with one click
- Delete and rename templates
- Template includes field names, selectors, types, and pagination settings

 **Files:** `utils/storage.js` | `options/options.html` | `options/options.js`

 **Done When:** Scrape Amazon with 3 fields → save as 'Amazon Products' → close extension → reopen on Amazon → template is suggested → load it → fields ready instantly

 **Beginner Note:** Use `chrome.storage.local` to save templates as JSON objects. Match templates to URLs using simple string matching (e.g. URL contains 'amazon.com').

M14: Progress Controls — Pause, Resume, Stop

 **Goal:** During scraping, user has full control with Pause, Resume, and Stop buttons and a live progress bar.

 **Why it matters:** Long scraping sessions need control. User may want to stop early or pause to check data.

Deliverables

- Progress bar shows percentage complete
- Live counter: 'Page 4 — 320 items collected'
- Pause button halts scraping mid-session
- Resume button continues from where it stopped
- Stop button ends scraping and keeps collected data
- Last scraped item preview shown during scraping

 **Files:** `popup/popup.html` | `popup/popup.css` | `popup/popup.js` | `background/scrapper-engine.js`

 **Done When:** Start scraping 10-page site → click Pause at page 3 → data stops → click Resume → continues from page 3

 **Beginner Note:** Use a shared state object (`isPaused`, `isStopped`) in the service worker. Check this flag at each pagination step before proceeding.

M15: Final UI Polish & Testing

 **Goal:** Full UI cleanup, error handling, edge case fixes, and testing on 10+ different websites.

 **Why it matters:** A working tool is not the same as a polished tool. This milestone makes it production-ready to deliver to client.

Deliverables

- Clean, professional popup UI design
- Error messages for common failures (selector not found, page timeout)
- Loading spinners and status indicators
- Tooltips and help text for new users
- Tested on: Amazon, eBay, Shopify, WooCommerce, Facebook, Instagram, LinkedIn, news sites, job boards, real estate sites

- No crashes or data loss on any tested site
- Extension package ready for Chrome Web Store or private distribution

 **Files:** All files — final review

 **Done When:** Give to a non-technical person — they can use it without any instructions

 **Beginner Note:** Do not skip this milestone. Most beginner projects fail here. Test on sites you did not build for — they will always break something.

Recommended Build Order

Follow this exact order. Each group represents a logical checkpoint where the extension is in a testable, working state.

Group 1 — Foundation (M1-M2)

Extension loads in Chrome. Content script injected into websites. Nothing scraped yet but the base is ready.

Group 2 — Selection (M3-M5)

User can hover, see XPath, and click to select fields. First interactive milestone. Show this to the client as early proof of progress.

Group 3 — Core Scraping (M6-M8)

Data is actually extracted. Live preview table shows results. Columns can be renamed and deleted. This is the MVP — a usable product.

Group 4 — Advanced Scraping (M9-M11)

Unlimited scroll, pagination, and AI auto-detection added. Extension now works on Instagram, Facebook, Amazon, and WooCommerce without manual setup.

Group 5 — Completion (M12-M15)

Export in all formats. Template system. Progress controls. Final polish and testing. Ready to deliver to client.

Tips for Beginners

1. Test After Every Milestone

Do not write code for 3 milestones and then test. Test after every single milestone. Bugs caught early take 10 minutes to fix. Bugs caught late take days.

2. Use Chrome DevTools Constantly

Right click any website → Inspect → Console. This is where you will see errors, test selectors, and debug. Get comfortable here before starting.

3. Message Passing is the Hardest Part

Chrome extensions have 3 separate environments (popup, content script, service worker) that cannot directly access each other. They communicate via `chrome.runtime.sendMessage()`. Understand this before M2.

4. Start Simple, Add Complexity

In M6 (data extraction), first make it work for text only. Then add links, images, prices. Do not try to handle all types at once.

5. Use Chrome Extension Docs

developer.chrome.com/docs/extensions is the official guide. It has examples for every API used in this project.

Complete File Structure

This is the full folder structure you will build across all 15 milestones:

```
web-scraper-extension/ |— manifest.json          ← M1 |— icons/
← M1 |— popup/ | |— popup.html          ← M1, M5, M7, M8, M12, M14 | |—
popup.css          ← M1, M3, M7, M8, M14, M15 | |— popup.js          ← M1,
M5, M7, M8, M12, M14 |— content/ | |— content.js          ← M2, M3, M4, M5, M6,
M9, M10, M11 | |— highlighter.css          ← M3, M4 |— background/ | |— service-
worker.js          ← M2, M14 | |— scraper-engine.js          ← M6, M9, M10, M14 |— utils/
| |— storage.js          ← M5, M13 | |— exporter.js          ← M12 | |—
dom-parser.js          ← M6, M11 | |— xpath-helper.js          ← M4 |— options/ | |—
|— options.html          ← M13 | |— options.js          ← M13 |— lib/ | |—
xlsx.min.js          ← M12
```

Next Steps

- Download this document and keep it open while developing
- Set up your code editor (VS Code recommended)
- Read Chrome Extension getting started guide: developer.chrome.com/docs/extensions/get-started
- Create your project folder and start Milestone 1
- Check off each deliverable as you complete it

Start with M1. Do not skip ahead. Each milestone is a building block — skip one and the next will not work.